



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра системного программирования

Егоров Илья Георгиевич
группа 627

Суперкомпьютерное моделирование и технологии
**Гибридная MPI/OpenMP многопоточная реализация
решения гиперболического уравнения в частных производных
на трехмерном прямоугольном параллелепипеде**

ОТЧЁТ

Москва, 2025, 23 Октября

Содержание

1	Описание задания и программной реализации	3
1.1	Постановка задачи	3
1.2	Численный метод решения задачи	3
1.3	Программная реализация	4
2	Исследование производительности	6
2.1	Характеристики вычислительной системы	6
2.1.1	Сравнение времени вычислений на GPU относительно CPU	6

1 Описание задания и программной реализации

1.1 Постановка задачи

В трёхмерной замкнутой области $\Omega = [0, L_x] \times [0, L_y] \times [0, L_z]$ для $t \in [0, T]$ требуется найти решение $u(x, y, z, t)$ задачи

$$\begin{cases} u_{tt}(x, y, z, t) = a^2 \Delta u(x, y, z, t) \\ u(x, y, z, 0) = \phi(x, y, z) \\ u_t(x, y, z, 0) = 0 \\ u(0, y, z, t) = u(L_x, y, z, t) = 0 \\ u(x, 0, z, t) = u(x, L_y, z, t) \\ u(x, y, 0, t) = u(x, y, L_z, t) = 0 \end{cases} \quad (1)$$

где $u(x, y, z, t) = \sin \frac{\pi x}{L_x} \sin \frac{2\pi y}{L_y} \sin \frac{3\pi z}{L_z} \cos(a_t t)$, $a_t = \frac{\pi}{2} \sqrt{\frac{1}{L_x} + \frac{4}{L_y} + \frac{9}{L_z}}$ — точное аналитическое решение, $\phi(x, y, z) = u(x, y, z, 0) = \sin \frac{\pi x}{L_x} \sin \frac{2\pi y}{L_y} \sin \frac{3\pi z}{L_z}$ и $a^2 = \frac{1}{4}$.

1.2 Численный метод решения задачи

Пусть заданы $N, K \in \mathbb{N}$, тогда пусть $h_x = \frac{L_x}{N}$, $h_y = \frac{L_y}{N}$, $h_z = \frac{L_z}{N}$, $\tau = \frac{T}{K}$ и $\omega_{h\tau} = \overline{\omega_h} \times \omega_\tau$, где

$$\overline{\omega_h} = \{(x_i, y_j, z_k) | x_i = ih_x, y_j = jh_y, z_k = kh_z, i, j, k \in \overline{0, N}\}$$

$$\omega_\tau = \{t = n\tau, n \in \overline{0, K}\}$$

Пусть $\omega_h = \text{int} \overline{\omega_h}$, т.е. внутренние узлы сетки.

Пусть $u_{ijk}^n = u(x_i, y_j, z_k, t_n)$, $\phi_{ijk} = \phi(x_i, y_j, z_k)$

Тогда

$$\begin{aligned} u_{0jk}^n &= u_{Njk}^n = u_{ij0}^n = u_{ijN}^n = 0, & i, j, k \in \overline{0, N}, n \in \overline{0, K} \\ u_{i0k}^n &= u_{iNk}^n, u_{i1k}^n = u_{iN+1k}^n, & i, k \in \overline{0, N}, n \in \overline{0, K} \\ u_{ijk}^0 &= \phi_{ijk} = \phi_{ijk} = \phi(x_i, y_j, z_k), & (x_i, y_j, z_k) \in \omega_h \\ u_{ijk}^1 &= u_{ijk}^0 + \frac{a^2 \tau^2}{2} \Delta_h \phi_{ijk} & (x_i, y_j, z_k) \in \omega_h \\ u_{ijk}^{n+1} &= 2u_{ijk}^n - u_{ijk}^{n-1} + \Delta_h u_{ijk}^n & (x_i, y_j, z_k) \in \omega_h \end{aligned}$$

Здесь Δ_h — семиточечный разностный аналог оператора Лапласа:

$$\Delta_h u_{ijk}^n = \frac{u_{i+1jk}^n - 2u_{ijk}^n + u_{i-1jk}^n}{h_x^2} + \frac{u_{ij+1k}^n - 2u_{ijk}^n + u_{ij-1k}^n}{h_y^2} + \frac{u_{ijk+1}^n - 2u_{ijk}^n + u_{ijk-1}^n}{h_z^2}$$

Будем рассматривать случай, когда $L_x = L_y = L_z = L$ и, соответственно, $h_x = h_y = h_z = h$. Для устойчивости решения требуется

$$\tau < h$$

Или же

$$\gamma = \frac{\tau}{h} < 1$$

Тогда при заданных L, N, K можно положить $T = \tau K = \gamma h K = \frac{\gamma L K}{N}$.

1.3 Программная реализация

Программа реализована на языке C++ (с++11/с++20). Для записи решения написан шаблонный класс *Grid*, который абстрагирует взаимодействие с памятью и индексированием. Также написаны функциональные классы *AnalyticalFunction* для инкапсуляции вычисления значения аналитической функции, *BoundaryFunction* для инкапсуляции вычисления значения граничных условий и *FunctionGridView*, — обёртка, делающая то же самое, но в терминах узлов сетки, и *InitialGridView* — для вычисления начальных условий. Для непосредственного решения написан абстрактный класс *Solver* с методом *solve*, возвращающий решение, — объект класса *Grid*, значение погрешности и времени, затраченного на вычисление решения и ошибки. Егор реализации, *CPU SolverImpl* и *GPU SolverImpl*, соответствуют CPU-only и GPU-only решениям.

При реализации функционала параллелизма с распределённой памятью были добавлены классы *Communicator* и *CubeCommunicator*, вспомогательные перечислимые типы *Edge* и *Tag* и производные от них, упрощающие межпроцессорное взаимодействие, а также класс *SolverFactory* для более удобного создания объектов класса *Solver* для конкретного процесса с его собственными параметрами. Также была использована асинхронная оптимизация — обмен данными и вычисления происходят одновременно: сначала инициализируется неблокирующий обмен гало и интерфейсными узлами, затем

происходят вычисления на внутренних узлах, после чего процессы проходят барьер на обмен и проводят вычисления на интерфейсных узлах.

Для CUDA-вычислений был написан специальный класс *DevicePoolManager*, которому делегирована работа с GPU-памятью, а также имеющий внутреннюю очередь регионов памяти на кольцевом буфере для оптимизации вычислений "по шагам и производный от него *CommDevicePoolManager*. Так же для упрощения взаимодействия с узлами сетки, буферами ошибок и гало и сущностями *Solver* были добавлены trivially copyable/standard layout классы *DeviceView*, *CommDeviceView* и *View*.

В качестве аргументов программа получает на вход N , а также px , py и pz — параметры разбиения сетки по процессам вдоль соответствующих осей. Значение K согласно условиям равно 20, а $L = 1$ и π .

2 Исследование производительности

2.1 Характеристики вычислительной системы

Имя: ПВС «IBM Polus»

Пиковая производительность: 55.84 TFlop/s

Производительность (Linpack): 40.39 TFlop/s

Вычислительных узлов: 5

На каждом узле:

Процессоры IBM Power 8: 2

NVIDIA Tesla P100: 2

Число процессорных ядер: 20

Число потоков на ядро: 8

Оперативная память: 256 Гбайт (1024 Гбайт узел 5)

Коммуникационная сеть: Infiniband / 100 Gb

Система хранения данных: GPFS

Операционная система: Linux Red Hat 7.5

2.1.1 Сравнение времени вычислений на GPU относительно CPU

Измерения, представленные на табл. 1 и 2, проводились по следующим правилам: для вычислений на CPU использовалось 20 MPI процессов, каждый по 8 OpenMP потоков (всего 160 потоков), для GPU использовался один MPI процесс с 1 и 2 графическими картами. На таблицах хорошо видно ускорение вычислений на GPU относительно CPU, а также линейный рост ускорения при увеличении количества карт.

Node Number per Side	Calc Mode	GPU Number	Solve Time (sec)	Calc Inner Node Time (sec)	Calc Boundary Node Time (sec)	Pack Time (sec)	Wait Time (sec)	Load Time (sec)	Store Time (sec)	Error
256	CPU	-	3.32307	2.10899	0.520225	0.136656	0.346268	-	-	2.01E-06
	GPU	1	0.418486	0.101363	0.00844199	8.85E-06	1.73E-05	1.15E-06	0.295921	2.01E-06
		2	0.429449	0.120516	0.012258	0.0534782	0.0277646	0.00126348	0.214548	2.01E-06
512	CPU	0	15.4312	12.33	1.89578	0.384228	1.20452	-	-	1.26E-07
	GPU	1	3.5922	1.31361	0.0587821	6.66E-06	1.87E-05	1.15E-06	2.16008	1.26E-07
		2	2.88123	1.23359	0.0736369	0.0330095	0.00983723	0.00350824	1.49823	1.26E-07

Таблица 1: Зависимость времени решения для $L = 1$

Node Number per Side	Calc Mode	GPU Number	Solve Time (sec)	Calc Inner Node Time (sec)	Calc Boundary Node Time (sec)	Pack Time (sec)	Wait Time (sec)	Load Time (sec)	Store Time (sec)	Error
256	CPU	0	3.12714	1.98583	0.507452	0.171536	0.272004	0	0	2.01E-06
	GPU	1	0.416419	0.100282	0.00807401	6.26E-06	1.23E-05	1.24E-06	0.295411	2.01E-06
		2	0.414669	0.127069	0.0144195	0.0473565	0.015395	0.00100458	0.197964	2.01E-06
512	CPU	0	8.94778	7.08004	1.52303	0.0102322	1.22896	-	-	1.26E-07
	GPU	1	3.46696	1.29283	0.0583615	7.12E-06	1.50E-05	1.56E-06	2.06933	1.26E-07
		2	2.8716	1.23962	0.0732759	0.0288002	0.00913294	0.00330745	1.5046	1.26E-07

Таблица 2: Зависимость времени решения для $L = \pi$